

Thinking Network

Foundational Web3 × AI Infrastructure

MOI BETA MAINNET LAUNCH - OCT 2025

THE BIG SHIFT

We are moving into a Digitally Interacting World

Not just connected by technology — but living through it.

THE PROBLEM TODAY

Today's technology is built for automation and connectivity.

It manages information — the where, what, and how.

THE EMERGING NEED

But our world is now about value transfers between participants

Not just information flowing through pipes.

THE WHY OF MOI

Why another L1?

Because current chains optimize the wrong problem.

THE FLAW OF TODAY'S CHAINS

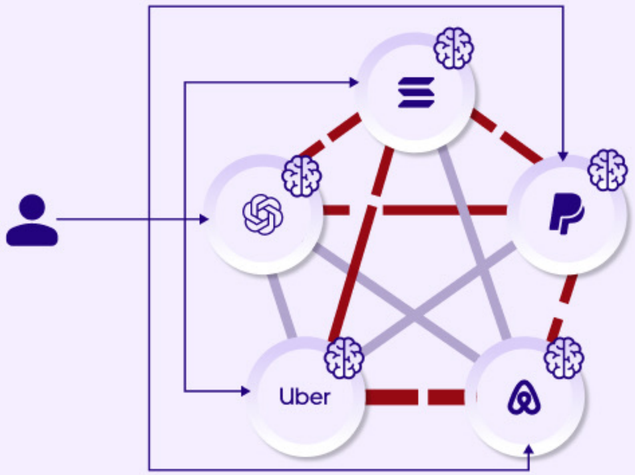
Today's blockchains force everything into global ordering.

They optimize for data replication, not value transfer.

The trilemma is a symptom, not the disease.

Contextual Compute introduces the Who — the fourth dimension of computing.

Today's Internet



Thinking Networks



Assets and intelligence live at the **network layer, not in apps.**
Networks evolve into **Thinking Networks.**

Real-world value transfer happens in contextual causal boundaries.



Your cab ride in NYC doesn't need global consensus with someone hiring a cab in SFO.



A trader swapping tokens shouldn't wait for someone buying coffee at Starbucks.

CONTEXTUAL COMPUTE

In MOI, Participants bring the right context to each interaction

Assets, preferences, network locus.

TRUE UNLIMITED PARALLELISM

Millions of independent contexts run in parallel.

No vulnerable shared state. No inter-operability bottlenecks.

Hyper-local finality with global connectivity.

THE AHA MOMENT

This is the HTTP moment for the Internet of Value.

Safe assets. Trusted agents. Seamless enterprise integration.

A true trust and interoperability layer for a million-node economy.

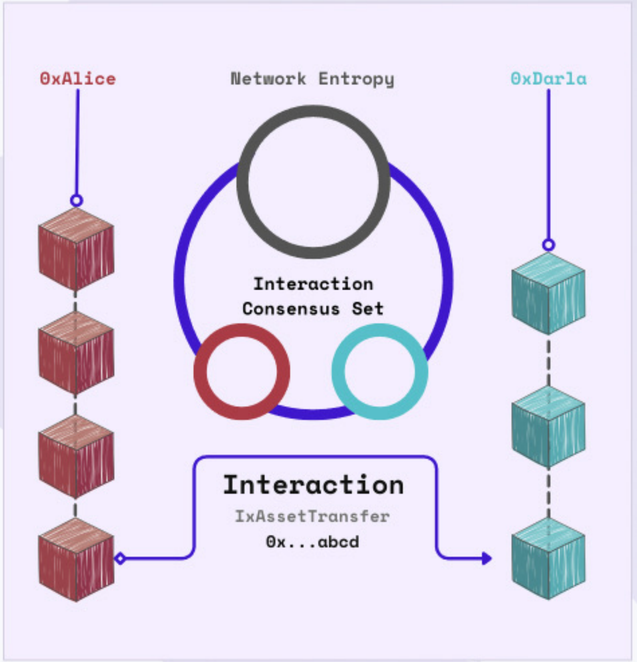
WHAT THIS ENABLES

- **Safe DeFi ($130B):** No honey pots. Unified Liquidity
- **Trusted Agentic AI ($100B+):** Agents gain persistent participant context.
- **Enterprise Integration ($80B+):** On-demand Trust. Predictable Costs.
- **Gaming ($65B+):** True asset ownership. Seamless cross-platform economies.

Context Object & Casual Ordering

Context Object

- **Assets**
- **Files**
- **Logics**
- **Trust**



TECH UPDATE

Core Features

- **Native Assets**
- **Context Inheritance**
- **Modulated Trust**
- **Composable Interactions**
- **Actor Scoped State**
- **Tallying Cost Model**



Native Assets

Multi Standard Support

MASO - Fungible Token

MAS1 - Non Fungible Token

MAS2 - Security Token

MAS3 - Dual Token



Context Inheritance

- **Context scoped operation**
- **Seamless Interoperability**



Modulated Trust

Fine tune safety and performance

Instant finality



Composable Interactions

Domain specific Interaction Ops - (Finance, SupplyChain, ERP)

Plug & Play Interface



Actor Scoped State

State Less Logics

Fine grain access control



Tallying Cost Model

Seperate pricing for storage and compute

Tallying rule based price updates

Content

- JavaScript SDK for MOI
- Coco Programming Language
- Coco Examples



Javascript SDK

MOI can be directly accessed through HTTP requests, JS SDK helps handling the requests

Domain-specific SDKs enable using MOI with minimal code: e.g., create an asset with a single call

JS SDK can be used to deploy logics, created by Coco or even manually generate manifests

SDK contains utilities for preparing and signing requests, communicating with providers (gateways to MOI network) and handling receipts

Documents on
https://js-moi-sdk.docs.moi.technology

```ts
import { OpType } from "js-moi-utils";
import { Signer } from "js-moi-signer";
import { InteractionContext } from "js-moi-wallet";
```

Gokul R, 3 days ago | 1 author (Gokul R)

```ts
export declare class MAS0AssetLogic {
    assetId: string;
    signer: Signer;
    constructor(assetId: string, signer: Signer);
    private polorize;
    static newAsset(signer: Signer, symbol: string, supply: number | bigint, manager: string, enableEvents: boolean): Promise<MAS0AssetLogic>;
    static create(signer: Signer, symbol: string, supply: number | bigint, manager: string, enableEvents: boolean): InteractionContext<OpType.ASSET_CREATE>;
    mint(beneficiary: string, amount: number | bigint): InteractionContext<OpType.ASSET_INVOKE>;
    burn(amount: number | bigint): InteractionContext<OpType.ASSET_INVOKE>;
    transfer(beneficiary: string, amount: number | bigint): InteractionContext<OpType.ASSET_INVOKE>;
    approve(beneficiary: string, amount: number | bigint, expiresAt: number): InteractionContext<OpType.ASSET_INVOKE>;
    revoke(beneficiary: string): InteractionContext<OpType.ASSET_INVOKE>;
    lockup(beneficiary: string, amount: number | bigint): InteractionContext<OpType.ASSET_INVOKE>;
    release(benefactor: string, beneficiary: string, amount: number | bigint): InteractionContext<OpType.ASSET_INVOKE>;
}
```

Gokul R, 4 days ago • feat: implemented asset and participant interac...



COCO

- Coco is a programming language for writing logics on MOI
- Highly readable, indentation-based
- Supports MOI concepts like logic vs. actor storage and native assets
- Enables writing custom asset logics, used when assets are created
- Documents on **https://cocolang.dev/**

Tour
Getting Started
Flipper: our first Coco program
Context-aware Flipper
Coco Language
Compiling and Running
Glossary

Flipper: our first Coco program

In this example, we're going to go step by step from installing Coco compiler to writing, compiling and running a small logic. The logic, called `Flipper` demonstrates the use of Coco compiler and test environment Cocolab. The only tool that's needed for the whole example is a `coco` binary one can install as described in installation instructions.

Create module

Coco module is the basic source code that compiles to a manifest that can be deployed to MOI. It contains a `coco.nut` file that serves as a configuration for module compilation, containing amonng other module name, output name, output options etc.

To create a module, first create a new folder, then run an `nut init` command in it.

```
mkdir flipper
cd flipper
coco nut init Flipper
```

`nut init` command creates a file `coco.nut` in the current folder that specifies `Flipper` as the module name, `flipper.yaml` as the output and PISA v0.5.0 in BIN format as the output. You can manually edit this file and most of the options in `coco.nut` file can be overridden with `coco` command options.

Write Coco code

Now let's write our first program. Coco module is a collection of all files in a folder with `.coco` extension and with `coco <module_name>` (in our example, `coco Flipper`) in the first line of code. As our example is simple, we'll just create one file, called `flipper.coco`:

flipper.coco

```
coco Flipper

state logic:
  values Map[Identifier]Bool
```



Coco Examples

github.com:sarvalabs/cocolang-examples.git

Short demo of creating and using assets

COMMUNITY

University Grant Program

- **12 week program**
- **Cash + Token Bounty $$$**
- **Direct support from MOI engineers**
- **Purdue University and PCCOE part of first cohort**

COMMUNITY

Community Channels

Telegram channel

AdithyaGanesh03

MOI Official Telegram channel

moi_tech

Discord channel

6zjT96UPdh

Thank You

- Join the MOI Beta Mainnet
- Join Discord

Join now